AgriHGT: Heterogeneous Graph Transformer for Cross-Crop Leaf Disease Relation Modeling in Tomato, Papaya, Zucchini, and Bottle Gourd

I. Introduction

Plant diseases remain a major challenge to agricultural productivity, crop quality, and food security. Automated diagnosis from leaf images offers a practical means of supporting farmers when timely access to plant pathology expertise is limited. Deep learning has substantially advanced image-based plant disease recognition, particularly on controlled datasets such as PlantVillage. Early studies applied convolutional neural networks (CNNs), sometimes together with conventional image-processing operations, to classify healthy and diseased leaves [1]. Deeper CNN architectures, including AlexNet, GoogLeNet, and VGG variants, subsequently achieved classification accuracy above 99% on large collections of plant-disease images [2].

More recent studies have investigated Vision Transformers (ViTs), which use self-attention to model global relationships among image patches [3], [4]. Hierarchical transformer architectures, such as Swin Transformer, have also been adapted to plant disease recognition under more challenging field-image conditions [5]. In parallel, Graph Neural Networks (GNNs) have been used to represent spatial or structural relationships among regions within individual leaf images [6], [7]. Other approaches combine segmentation networks, CNN-transformer classifiers, and metaheuristic optimization within heterogeneous ensemble pipelines [8].

Despite these advances, most existing methods still formulate plant disease recognition as an independent image-classification problem. Relational information connecting crops, diseases, coarse disease categories, and visually similar leaf images is rarely represented explicitly within a unified model. Existing image-based GNN approaches generally construct graphs from regions within an individual image or consider a restricted collection of crops. Likewise, heterogeneous ensemble methods improve classification through multiple processing stages rather than through an explicit graph of crop–disease relationships. Consequently, the potential value of heterogeneous relational modelling in joint multi-crop disease classification remains insufficiently investigated.

This paper proposes AgriHGT, a two-stage multi-crop plant disease classification framework based on the Heterogeneous Graph Transformer (HGT) of Hu et al. [9]. In the first stage, a pretrained visual backbone is fine-tuned using the training partition and then used to extract fixed image embeddings. In the second stage, these embeddings are assigned to leaf-image nodes in a heterogeneous graph containing four node types: `leaf_image`, `crop`, `disease`, and `disease_category`. Node- and relation-type-dependent attention is then used to model interactions among these entities.

The graph contains crop-membership, crop–disease, disease-category, image-similarity, and conditionally available image–disease relations. The `similar_to` relation connects each leaf image to its nearest neighbours in backbone feature space within the same crop. Disease nodes that belong to the same coarse category are also connected to provide an explicit structural path between related conditions occurring on different host plants. These relations are designed to support joint cross-crop relational reasoning; however, the extent to which they improve classification is evaluated empirically rather than assumed.

A central challenge is that connecting every labelled training image directly to its disease node can create a structural shortcut in a multi-layer message-passing model. AgriHGT addresses this problem by dividing the training images into two fixed, disjoint subsets. Images in `train_msg` retain their image–disease edges and provide visual information to disease-node representations, but they do not contribute to the classification loss. Images in `train_sup` have no image–disease edge and are the only training images used to calculate the loss. Their graph structure therefore matches that of validation and test images. The main objectives of this study are as follows:

- To construct a heterogeneous graph containing leaf images, crops, diseases, and disease categories.
- To create a duplicate-aware and support-controlled dataset split with at least one validation and one test image for every class.
- To evaluate seven visual backbones: ResNet50, ConvNeXt-Tiny, EfficientNet-B3, Swin-Tiny, DenseNet121, DINOv2 ViT-B/14, and DINOv3 ViT-B/16.
- To compare zero-, one-, and two-layer HGT models in order to examine the effect of graph message passing.
- To compare AgriHGT with feature-only logistic regression.
- To perform ablation experiments to examine the importance of graph relations, metadata nodes, residual features, and the learned fusion gate.

The experiments are conducted in a transductive graph setting: the features and graph connections of validation and test images are available during message passing, but their labels are excluded from training and gradient computation. The experimental results and comparisons are reported in Section IV.

II. Related Work

A. CNN-Based Plant Disease Detection

Convolutional neural networks (CNNs) have been widely used for image-based plant disease classification because of their ability to learn discriminative colour, texture, and shape features directly from leaf images. Srivastava et al. [1] presented a CNN-based plant disease recognition approach using a collection of healthy and diseased leaf images. Their work demonstrated the feasibility of replacing manually designed image features with representations learned directly from training data. Ferentinos [2] evaluated several CNN architectures, including AlexNet, GoogLeNet, and VGG variants, on 87,848 images representing 58 plant–disease combinations from 25 plant species. The best-performing model achieved 99.53% accuracy on previously unseen test images.

These studies established CNNs as effective feature extractors for plant disease recognition. However, their predictions are produced from the visual content of each image independently. Although multiple crops and diseases may be included in the same dataset, their semantic relationships are not explicitly represented during learning. For example, two diseases that produce related symptoms on different crops are treated only as separate output classes rather than as connected entities in a relational structure.

B. Vision Transformer and Swin Transformer Approaches

Vision Transformers (ViTs) extend image classification by dividing an image into patches and using self-attention to model long-range interactions among them. Murugavalli and Gopi [3] proposed PLA-ViT, which combines patch-based self-attention, data augmentation, and adaptive learning-rate scheduling for plant leaf disease classification. Boukabouya et al. [4] compared multiple ViT configurations on a tomato leaf disease dataset derived from PlantVillage and reported accuracy as high as 99.7%.

Liu and Zhang [5] proposed an Efficient Swin Transformer containing a Selective Token Generator and Feature Fusion Aggregator. Their method reduced the parameter count by approximately 20.89% relative to standard Swin-T while improving plant disease recognition under field-image conditions. Parez et al. [10] introduced GreenViT, an efficient ViT-based classifier evaluated on PlantVillage, the Data Repository of Leaf Images, and a merged Plant Composite dataset. The model achieved reported accuracies of 100%, 98%, and 99%, respectively.

Hybrid CNN–ViT architectures have also been investigated. Shandilya et al. [11] combined a CNN branch for local feature extraction with a ViT branch for long-range contextual modelling in maize leaf disease classification. Singha Roy and Kukreja [12] used a shared ViT backbone to jointly predict rice disease type and disease severity from a custom dataset of 3,345 images. Their model obtained weighted F1-scores of 54.17% for disease classification and 77.94% for severity estimation.

These transformer-based approaches improve the representation of spatial relationships within an image. Nevertheless, they do not explicitly represent crops, diseases, or disease groups as graph entities. Their self-attention operates primarily among patches or feature tokens from the same image, rather than among typed entities distributed across multiple crop species.

C. Graph Neural Network-Based Approaches

Graph Neural Networks (GNNs) provide a mechanism for learning from non-Euclidean structures in which nodes communicate through predefined or learned edges. In plant disease recognition, most existing GNN applications construct graphs from image regions or use graph operations to aggregate local visual features.

Ananthajothi and Lokesh [6] constructed image-level graphs to capture structural relationships among leaf regions for plant disease diagnosis. Elumalai et al. [7] proposed an Attention-Enhanced GNN for fungal disease classification using monospectral leaf images. Their model used graph attention to emphasize disease-relevant regions and was evaluated on spinach and curry-leaf images.

Bera et al. [13] proposed PND-Net, in which a graph convolutional network is placed above a CNN backbone. Region-based and spatial-pyramid features are represented as graph nodes to improve plant nutrition-deficiency and disease classification. The model was evaluated on several datasets and achieved reported accuracies ranging from 84.30% on PlantDoc to 96.18% on a potato disease dataset.

Bajpai et al. [14] combined a ResNet18 branch with a GNN branch for potato leaf disease recognition. Maheswara Rao et al. [15] integrated CNN and graph convolutional components for disease classification across 31 classes from eight crops. Although the latter study includes several crops, it does not construct a heterogeneous entity graph in which crop, disease, and image nodes are represented as different node types.

Surekha and Subha Mastan Rao [16] proposed AdjLeafGNN, which combines an enhanced CNN with a GNN to perform multi-class disease classification and estimate disease-spread probability between adjacent leaves. This extends graph modelling beyond individual leaf-region classification, but the graph is designed around physical leaf adjacency rather than semantic relationships among crop, disease, and disease-category entities.

Collectively, these studies demonstrate that graph-based aggregation can improve the representation of spatial, regional, or adjacency relationships. However, most of their graphs are homogeneous or task-specific. They do not employ a typed heterogeneous graph containing separate leaf-image, crop, disease, and disease-category nodes with relation-specific message passing.

D. Knowledge-Graph and Graph-Related Attention Approaches

Knowledge graphs provide an alternative way to encode agricultural entities and their semantic relationships. Unlike image-region GNNs, these methods frequently obtain their graph structure from textual documents, expert knowledge, or external databases.

Li et al. [17] proposed MARBC, a RoBERTa–BiLSTM–CRF model with multi-head attention for extracting entities and relations from a Chinese rice disease and pest corpus. The extracted information was used to construct a heterogeneous agricultural knowledge graph, with a reported F1-score of 94.44% for the information extraction task. The method organizes textual agricultural knowledge but does not directly classify leaf images through heterogeneous image-node message passing.

Zhao et al. [18] combined a large language model, an agricultural knowledge graph, and a graph attention network for disease detection in *Elaeagnus angustifolia*. Their results show that external structured knowledge can support plant disease analysis. However, the study focuses on one plant species and differs from a multi-crop image graph in which each observed leaf is represented as a graph node.

Lin et al. [19] proposed GPA-Net, a pyramid neural network containing graph-related attention modules for fine-grained disease and pest identification. It learns multiscale spatial and graph-inspired feature relationships and was evaluated on cassava leaf, AI Challenger, and IP102 datasets. Although GPA-Net employs graph-related attention, it remains primarily a visual feature-learning architecture rather than an explicit heterogeneous knowledge graph of crop, disease, and image entities.

These studies demonstrate the usefulness of structured agricultural knowledge and graph-inspired attention. Nevertheless, a gap remains between text-oriented agricultural knowledge graphs and image-oriented graph models. Few approaches directly integrate image embeddings with explicit crop and disease entities in a single heterogeneous message-passing architecture.

E. Heterogeneous Ensemble and Hybrid Transformer Approaches

Kalpana et al. [8] proposed a heterogeneous ensemble for multiple plant disease diagnosis. Their pipeline first uses U-Net and Swin Transformer V2 to segment diseased regions. The segmented images are then classified using CoAtNet-based branches, and the branch predictions are combined using weights optimized by a Levy Flight Honey Badger Algorithm. The approach achieved a reported accuracy of 99.31%, precision of 99.32%, and F1-score of 99.31% on PlantVillage.

In this context, the term *heterogeneous* refers to the combination of different segmentation, classification, and optimization models. It does not refer to a heterogeneous graph containing multiple node and relation types. The model therefore improves individual-image classification through architectural diversity, but it does not provide an explicit representation of relationships among crops, diseases, disease categories, and individual leaf images. Its multiple segmentation, classification, and optimization stages also make it substantially different from a fixed-feature graph-classification pipeline.

F. Summary and Research Gap

The reviewed literature demonstrates substantial progress in plant disease recognition. CNN and transformer models learn strong visual representations, graph neural networks capture spatial or regional relationships, knowledge graphs organize agricultural entities, and ensemble architectures combine complementary image-processing models. However, these research directions generally remain separated.

CNNs and ViTs commonly treat each leaf as an independent classification instance. Existing image-based GNNs primarily construct graphs from regions, feature maps, or adjacent leaves. Knowledge-graph studies usually derive their entities from text or external agricultural knowledge, while heterogeneous ensemble models combine different neural architectures without constructing a heterogeneous entity graph. Although some studies include multiple crops, multi-crop classification alone does not explicitly model the semantic relations connecting crops, diseases, coarse disease groups, and visually similar leaf images.

A further limitation is that an improvement produced by a complex graph or attention model cannot be attributed to message passing unless it is compared with suitable non-relational and reduced-graph alternatives. Controlled comparisons between feature-only models, zero-layer graph models, and models with different message-passing depths are therefore important for determining whether relational propagation provides additional value beyond the visual backbone.

AgriHGT addresses this research gap through a two-stage heterogeneous relational classification framework. A pretrained backbone is first fine-tuned and used to extract fixed disease-discriminative image features. These features are then assigned to leaf-image nodes in a graph that also contains crop, disease, and coarse disease-category nodes. Heterogeneous Graph Transformer layers perform node-type- and relation-type-dependent message passing over this graph.

The framework is evaluated using zero-, one-, and two-layer configurations and a feature-only logistic-regression reference model. Duplicate-aware data preparation and a separate message-passing and supervision partition are used to reduce potential shortcut learning and split contamination. The experiments are conducted in a transductive setting in which validation and test node features participate in graph construction and message passing, while their labels remain excluded from model optimization. Consequently, the study evaluates whether typed relational modelling improves joint multi-crop classification without assuming in advance that cross-crop knowledge transfer has occurred.

III. Methodology

A two-stage framework was used for multi-crop leaf-disease classification. In the first stage, pretrained visual backbones were adapted to the Agri-Vision4 classification task and were subsequently used to extract fixed image embeddings. In the second stage, the embeddings were assigned to leaf-image nodes in a heterogeneous graph, and classification was performed using a Heterogeneous Graph Transformer (HGT).

The graph data structure and `HGTConv` operations were obtained from the official pyHGT implementation [9]. Additional components were implemented for duplicate-aware and support-controlled data splitting, learnable non-image

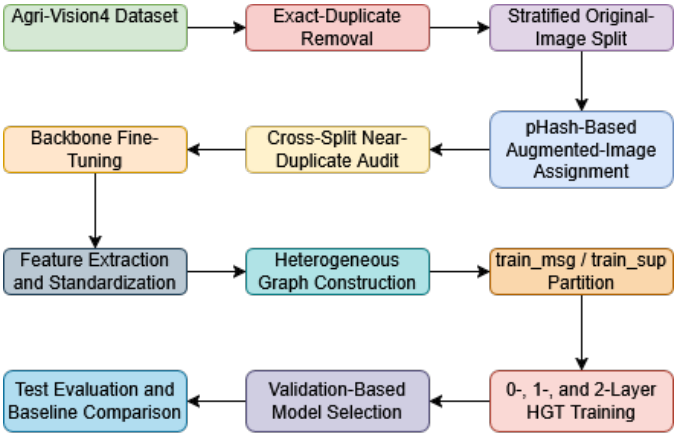


Fig. 1: Overview of the support-controlled AgriHGT methodology.

node embeddings, zero-layer message-passing ablation, gated residual fusion, and leaf–disease compatibility scoring. The complete methodological workflow is shown in Fig. 1.

A. Dataset and Task Formulation

The Agri-Vision4 dataset [20] contains leaf images from Tomato, Papaya, Zucchini, and Bottle Gourd. Twenty-eight crop-qualified disease and health classes are represented. The dataset contains an *Original* image pool and an *Augmented* image pool derived from the Original images.

The task was formulated as a unified 28-class classification problem. Each output class represents a disease or healthy condition associated with a particular crop. Image, crop, disease, and disease-category information could therefore be represented within a common heterogeneous graph.

The unified label space was not treated by itself as evidence of cross-crop knowledge transfer. The contribution of shared cross-crop structure was examined separately through graph-component ablations and comparisons between the complete multi-crop graph and independently trained crop-specific graphs.

B. Exact-Duplicate Removal

Exact duplicate removal was performed before the training–validation–test split was constructed. An MD5 hash was calculated for each of the 28,121 image files. Files with identical hashes were grouped, and one representative was retained from each group.

A total of 1,255 byte-identical duplicates were removed. The resulting dataset contained 26,866 images, consisting of 4,304 Original images and 22,562 Augmented images.

C. Duplicate-Aware Initial Split

A random split of the combined Original and Augmented image pools could place an Original image and one of its transformed derivatives in different partitions. Such overlap could result in optimistic validation and test performance. The Original images were therefore used as split anchors.

Class-stratified sampling with a 70/15/15 ratio was first applied to the Original-image pool. The initial assignment contained 3,004 training images, 650 validation images, and 650 test images from the Original pool.

Perceptual hashes were subsequently calculated using

$$\texttt{hash_size} = 16,$$

which produced 256-bit hashes. Each Augmented image was compared with Original images belonging to the same crop and class. When the minimum Hamming distance was no greater than six, the Augmented image was linked to the nearest Original image and inherited its partition.

Under this criterion, 1,552 Augmented images were linked to Original images. The remaining 21,010 Augmented images could not be linked confidently to a particular Original image and were initially assigned to the training partition.

D. Post-Hoc Near-Duplicate Audit

A same-class post-hoc audit was performed after the initial split had been constructed. Perceptual hashes were compared across different partitions for the following image-pair types:

- Original–Original;
- Original–Augmented; and
- Augmented–Augmented.

When a same-class pair from different partitions had a perceptual-hash Hamming distance of six or less, the validation- or test-side image was reassigned to the training partition. The audit was repeated until no additional cross-partition violation was detected.

Successive audit rounds identified 41, 11, 3, and finally 0 violating pairs.

E. Per-Class Support Control

The duplicate audit could remove the only validation or test image of a minority class. A support-control procedure was therefore applied after the audit.

When a class contained no validation image or no test image, an eligible training image from the same class was reassigned to the missing partition. The support-control procedure and same-class near-duplicate audit were then repeated in a bounded loop. A reassigned image could therefore be returned to training when a new cross-partition near-duplicate relation was detected.

The final support-controlled split contained

$$25{,}122 \text{ training images},$$

$$896 \text{ validation images},$$

and

$$848 \text{ test images}.$$

At least one validation image and one test image were retained for every class. The crop-level distribution was as follows:

- Bottle Gourd: 6,210 training, 300 validation, and 284 test images;
- Papaya: 6,201 training, 281 validation, and 281 test images;
- Tomato: 4,630 training, 186 validation, and 170 test images; and
- Zucchini: 8,081 training, 129 validation, and 113 test images.

The final Original-image counts were 3,035 training images, 634 validation images, and 635 test images.

The same final split was used for every backbone experiment. Therefore, differences among the backbone results were not caused by different training, validation, or test partitions.

The partition is described as *duplicate-aware* or *leakage-controlled*, rather than completely leakage-free. The audit was restricted to same-class comparisons and to similarities detected by the selected perceptual-hash representation and threshold. The supported claim is therefore that no same-class cross-partition near-duplicate pair was detected at a pHash Hamming distance of six or less.

F. Visual Backbone Selection

Seven visual backbones were evaluated using the same support-controlled split:

- ResNet50;
- ConvNeXt-Tiny;
- EfficientNet-B3;
- Swin-Tiny;
- DenseNet121;
- DINOv2 ViT-B/14; and
- DINOv3 ViT-B/16.

The selected backbones represent complementary visual-learning architectures. ResNet50 was selected as a residual convolutional network, while DenseNet121 was selected to represent dense feature reuse across convolutional layers. EfficientNet-B3 was selected to represent compound scaling of network depth, width, and input resolution.

ConvNeXt-Tiny was selected as a modern convolutional architecture influenced by design principles commonly associated with vision transformers. Swin-Tiny was selected as a hierarchical window-based vision transformer.

DINOv2 ViT-B/14 and DINOv3 ViT-B/16 were selected as self-supervised transformer encoders. Their pretrained representations were learned from large-scale image collections without the use of Agri-Vision4 plant-disease labels.

The selected set therefore covers residual convolution, dense connectivity, efficient compound scaling, modern convolution, hierarchical self-attention, and large-scale self-supervised transformer learning.

G. Backbone Adaptation

A temporary 28-class classification head was attached to each backbone. Most pretrained parameters were frozen, while the final representation stage and temporary classification head were made trainable.

For ResNet50, `layer4` and the fully connected classification head were unfrozen. For ConvNeXt-Tiny, `features[7]` and the classification head were unfrozen. For EfficientNet-B3, `features[7:]` and the classification head were unfrozen.

For Swin-Tiny, the final transformer stage, terminal normalization layer, and classification head were unfrozen. For DenseNet121, `denseblock4`, `norm5`, and the classification head were unfrozen.

For DINOv2 ViT-B/14, pretrained weights were loaded through the official Facebook Research `torch.hub` implementation

`dinov2_vitb14`.

The final two transformer blocks, terminal LayerNorm, and temporary classification head were made trainable.

For DINOv3 ViT-B/16, the `timm` implementation

`vit_base_patch16_dinov3.lvd1689m`

was used. The final two transformer blocks, available terminal normalization modules, and temporary classification head were made trainable. Gradient checkpointing was enabled when supported by the implementation.

H. Backbone Training Procedure

AdamW was used for backbone adaptation. ResNet50, ConvNeXt-Tiny, EfficientNet-B3, Swin-Tiny, and DenseNet121 were optimized using a learning rate of

$$1 \times 10^{-4}$$

and weight decay of

$$1 \times 10^{-4}.$$

A smaller learning rate of

$$5 \times 10^{-5}$$

was used for DINOv2 and DINOv3, while weight decay remained 1×10^{-4}. Gradient norms were clipped at 1.0.

Random resized cropping with a scale range of 0.8–1.0 and random horizontal flipping were applied during training. Validation images were processed using deterministic resizing and normalization. The checkpoint with the highest validation accuracy was retained for backbones trained with early stopping.

The backbone-specific settings are summarized in Table I.

Swin-Tiny and DenseNet121 were trained for a fixed 20-epoch schedule, whereas validation-based early stopping was used for the remaining backbones.

When a GPU out-of-memory error occurred, the batch size was reduced and training was restarted from the same initial pretrained checkpoint. Frozen normalization layers were retained in evaluation mode where applicable.

TABLE I: Backbone Adaptation and Feature-Extraction Settings

Backbone	Maximum Epochs	Patience	Batch Size	Input Size	Embedding Dimension
ResNet50	100	10	64	224×224	2,048
ConvNeXt-Tiny	400	20	32	224×224	768
EfficientNet-B3	400	20	32	300×300	1,536
Swin-Tiny	20	–	32	224×224	768
DenseNet121	20	–	64	224×224	1,024
DINOv2 ViT-B/14	400	20	32	224×224	768
DINOv3 ViT-B/16	400	20	32	256×256	768

I. Feature Extraction

After backbone adaptation, the temporary classification head was removed or replaced with an identity mapping. Each backbone was then frozen, and a deterministic feature-extraction pass was performed over all 26,866 images. Random augmentation was not applied during feature extraction.

The following feature matrices were produced:

- ResNet50: $26{,}866 \times 2{,}048$;
- ConvNeXt-Tiny: $26{,}866 \times 768$;
- EfficientNet-B3: $26{,}866 \times 1{,}536$;
- Swin-Tiny: $26{,}866 \times 768$;
- DenseNet121: $26{,}866 \times 1{,}024$;
- DINOv2 ViT-B/14: $26{,}866 \times 768$; and
- DINOv3 ViT-B/16: $26{,}866 \times 768$.

J. Feature Standardization

Feature standardization was performed independently for each backbone. Feature-wise means and standard deviations were estimated using training embeddings only. The resulting statistics were then applied unchanged to the training, validation, and test embeddings.

For feature x_j, standardization was performed as

$$\hat{x}_j = \frac{x_j - \mu_j^{(\text{train})}}{\sigma_j^{(\text{train})} + \epsilon},$$

where $\mu_j^{(\text{train})}$ and $\sigma_j^{(\text{train})}$ denote the mean and standard deviation estimated from the training embeddings, and ϵ denotes a small numerical constant.

A separate heterogeneous graph was constructed from the standardized feature matrix produced by each backbone.

K. Heterogeneous Graph Schema

Fig 2 shows the graph schema. Four node types were represented in every backbone-specific graph:

- `leaf_image`: one node for each image;
- `crop`: one node for each crop species;
- `disease`: one node for each crop-qualified disease or health class; and
- `disease_category`: one node for each coarse disease category.

Each graph contained 26,866 leaf-image nodes, four crop nodes, 28 disease nodes, and seven disease-category nodes. The total number of nodes was

$$26{,}905.$$

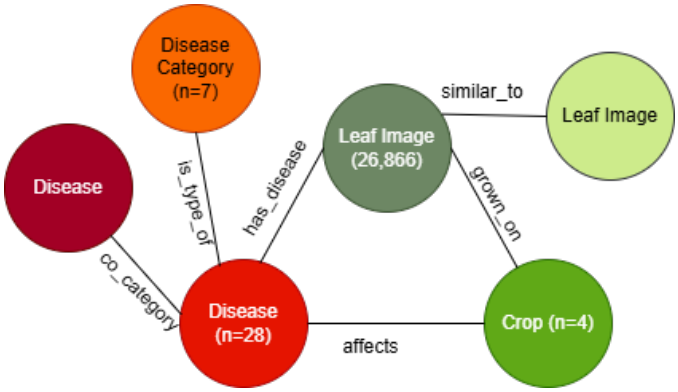


Fig. 2: Heterogeneous Graph Schema

The seven coarse disease categories were:

- Healthy;
- Viral;
- Bacterial;
- Fungal;
- Pest;
- Nutrient Deficiency; and
- Other.

Terms related to nutrient deficiency, including *iron* and *chlorosis*, were checked before the more general term *damage*. Classes containing *Xanthomonas* were assigned to the Bacterial category.

L. Node Initialization

Leaf-image nodes were initialized using the standardized embeddings produced by the corresponding visual backbone.

Crop, disease, and disease-category nodes did not have naturally defined continuous image features. Zero placeholder vectors with the same input dimension as the corresponding image embeddings were therefore assigned during graph construction.

During every model forward pass, the placeholder vectors were replaced with separately learnable meta-node embeddings. A distinct trainable identity representation was consequently provided for every crop, disease, and disease-category node.

M. Graph Relations

Six forward relation families were constructed:

1) `leaf_image` → `crop` using the `grown_on` relation;
2) `crop` → `disease` using the `affects` relation;

3) disease → disease_category using the is_type_of relation;
4) leaf_image → leaf_image using the similar_to relation;
5) leaf_image → disease using the has_disease relation; and
6) disease → disease using the co_category relation.

The co_category relation connected disease nodes assigned to the same coarse category. For example, viral diseases belonging to different crops could be connected directly. A one-hop cross-crop disease path was therefore provided in addition to the indirect path through a shared disease-category node.

All relation families were initially inserted as directed edges. Corresponding reverse relations were generated automatically by pyHGT. Forward and reverse relations were assigned separate relation identifiers and learnable parameters.

A self-loop was also added to every node during full-batch tensor conversion. Thirteen relation types were represented after the forward, reverse, and self-loop relations were included.

N. Image-Similarity Relation

The similar_to relation was generated independently for each backbone because the nearest-neighbour structure depended on the corresponding embedding space.

For every image, the five nearest neighbours were identified using cosine distance:

$$d_{\cos}(\mathbf{x}_i, \mathbf{x}_j) = 1 - \frac{\mathbf{x}_i^\top \mathbf{x}_j}{\|\mathbf{x}_i\|_2 \|\mathbf{x}_j\|_2}.$$

Neighbour search was restricted to images belonging to the same crop. Disease labels were not used during nearest-neighbour construction.

Five directed nearest-neighbour edges were created for every image, producing

$$26{,}866 \times 5 = 134{,}330$$

forward similar_to edges in each graph.

O. Transductive Evaluation Setting

Validation and test image nodes were included during construction of the similar_to graph. Their image features and graph connections were therefore available during model training. However, their disease labels were excluded from the optimization objective.

The reported setting is consequently transductive rather than inductive. The reported results measure classification of unlabelled nodes already present in the graph and should not be interpreted as classification of entirely new graph nodes introduced after training.

P. Message-Passing and Supervision Partition

A has_disease edge from every labelled training image to its disease node would expose the correct disease relation to the same image whose prediction contributed to the classification loss. After reverse relations were introduced, a shortcut could be formed through

$$\texttt{leaf_image} \rightarrow \texttt{disease} \rightarrow \texttt{leaf_image}.$$

The training partition was therefore divided into two disjoint roles. The division was performed independently within each class using a 75/25 message-passing-to-supervision ratio. An Original image and its linked Augmented derivatives were treated as one group and were assigned to the same role.

The same fixed partition was used for every backbone experiment. It contained

$$18{,}881$$

train_msg images and

$$6{,}241$$

train_sup images.

- train_msg images retained their has_disease edges. Their image representations contributed to disease-node representations through message passing, but their predictions were excluded from the classification loss.
- train_sup images were not assigned has_disease edges. Their predictions were the only training predictions included in the classification loss.

Validation and test images were also not assigned has_disease edges. The same true-label edge-availability rule was therefore applied to train_sup, validation, and test images.

The direct label shortcut was removed from supervised examples, while disease nodes remained visually grounded through the train_msg subset. The design does not constitute proof that no indirect information can propagate through other graph paths.

Q. Heterogeneous Graph Transformer

The relational classifier was based on the Heterogeneous Graph Transformer proposed by Hu et al. [9]. HGT applies node-type-specific transformations and relation-specific attention and message transformations.

Each node was first projected into a shared 256-dimensional hidden space using a node-type-specific linear layer. A hyperbolic tangent activation and dropout were then applied. Relation-specific parameters were used during heterogeneous message passing.

Four attention heads and an HGT dropout rate of 0.2 were used. Temporal encoding was disabled because no temporal information was present in the dataset.

Training was performed in full-batch mode. The complete graph was processed once during each epoch rather than being divided into sampled subgraphs.

R. Message-Passing Depth

Three HGT depths were evaluated for every visual backbone:

$$n_{\text{layers}} \in \{0, 1, 2\}.$$

A custom flexible implementation was used so that the zero-layer configuration skipped every HGT message-passing operation. The zero-layer model retained:

- node-type-specific input projections;
- learnable crop, disease, and disease-category embeddings;
- the gated residual projection; and
- the leaf–disease compatibility decoder.

The zero-layer configuration therefore provided a controlled no-message-passing reference.

S. Gated Residual Leaf Representation

Graph propagation could modify or suppress useful information already contained in the original visual embedding. A gated residual path was therefore used to combine the graph-propagated representation with a direct projection of the backbone feature.

Let $\mathbf{r}_i$ denote the normalized residual projection of the original feature for image i, and let $\mathbf{h}_i$ denote the corresponding HGT representation. A per-dimension gate was calculated as

$$\mathbf{g}_i = \sigma\left(W_g[\mathbf{r}_i \| \mathbf{h}_i] + \mathbf{b}_g\right),$$

where $\|$ denotes concatenation and $\sigma(\cdot)$ denotes the sigmoid function.

The final leaf representation was calculated as

$$\mathbf{z}_i = \mathbf{g}_i \odot \mathbf{r}_i + (1 - \mathbf{g}_i) \odot \mathbf{h}_i.$$

The gate bias was initialized to 2.0, corresponding to an initial sigmoid value of approximately 0.88. Greater emphasis was therefore placed initially on the visual residual, while the contribution of graph-propagated information could be adjusted during training.

T. Leaf–Disease Compatibility Decoder

Each final leaf representation was compared with all 28 disease-node representations using an MLP compatibility decoder.

Separate projections were applied to the leaf and disease representations:

$$\mathbf{u}_i = W_l \mathbf{z}_i + \mathbf{b}_l,$$

$$\mathbf{v}_j = W_d \mathbf{d}_j,$$

where $\mathbf{d}_j$ denotes the representation of disease node j.

The compatibility score between leaf i and disease j was calculated as

$$s_{ij} = W_o \,\text{Dropout}\left(\text{ReLU}(\mathbf{u}_i + \mathbf{v}_j)\right) + b_o.$$

A decoder hidden width of 128 and a decoder dropout rate of 0.1 were used. Twenty-eight logits were generated for every leaf image and were passed directly to the cross-entropy loss.

U. Main HGT Training Procedure

The complete transductive graph was included in every forward pass. Loss calculation was restricted to `train_sup` images. Labels from `train_msg`, validation, and test images were excluded from the loss.

AdamW was used with a learning rate of

$$1 \times 10^{-3}$$

and weight decay of

$$1 \times 10^{-4}.$$

Gradient norms were clipped at 1.0. A cosine-annealing schedule reduced the learning rate to

$$1 \times 10^{-6}$$

over the corresponding training budget.

Inverse-frequency class weights were estimated using `train_sup` labels only. The weights were normalized to a mean of one and were supplied to weighted cross-entropy.

The HGT training settings are summarized in Table II.

For each backbone, zero-, one-, and two-layer models were trained using the same fixed random seed and the same message/supervision partition.

The checkpoint with the highest validation accuracy, subject to a minimum improvement of 1×10^{-4}, was retained. Model depth was also selected using validation accuracy. Test accuracy and macro-F1 were calculated only after the selected checkpoint had been restored.

V. Feature-Only Reference Baseline

A multinomial logistic-regression classifier was trained separately for each backbone. The same standardized embeddings used by the corresponding AgriHGT model were supplied to the baseline.

Graph edges, metadata nodes, heterogeneous message passing, residual gating, and the leaf–disease compatibility decoder were not used by the baseline. All 25,122 training images were used. A maximum of 2,000 optimization iterations and the same fixed random seed were applied.

The logistic-regression comparison was treated as a practical feature-only reference. It was not treated as a complete isolation of message passing because the decoder, metadata nodes, node embeddings, and supervision procedure also differed. The zero-layer HGT comparison provided the more controlled message-passing ablation.

W. Multi-Seed ResNet50 Graph-Component Ablation

A multi-seed graph-component ablation was performed using the support-controlled split and ResNet50 feature bank. A one-layer HGT model was used to reduce the computational cost of the complete experiment grid.

Four graph configurations were evaluated:

1) the complete graph;
2) the graph without `co_category` edges;

TABLE II: Backbone-Specific HGT Training Settings

Backbone	Maximum Epochs	Patience
ResNet50	300	300
ConvNeXt-Tiny	400	50
EfficientNet-B3	400	50
Swin-Tiny	300	300
DenseNet121	300	300
DINOv2 ViT-B/14	400	50
DINOv3 ViT-B/16	400	50

3) the graph without disease-category nodes and `is_type_of` edges; and
4) the graph without `similar_to` edges.

The dataset split and ResNet50 feature bank were held fixed. Each configuration was trained using seeds

$$42,\ 43,\ 44,\ 45,\ 46.$$

A fresh message/supervision partition and fresh model initialization were generated for every seed. Variation across runs therefore reflected both optimization randomness and variation in the internal `train_msg`/`train_sup` assignment.

A maximum of 150 epochs and an early-stopping patience of 30 epochs were used. The hidden dimension remained 256, four attention heads were retained, dropout remained 0.2, and the nearest-neighbour setting remained $K = 5$.

Mean and standard deviation were calculated across the five runs.

X. DINOv3 Architectural Ablation

A comprehensive architectural ablation was conducted using DINOv3 ViT-B/16 embeddings. The support-controlled split, DINOv3 feature bank, model initialization seed, and optimization settings were held constant.

The HGT depth selected using validation accuracy in the main DINOv3 experiment was used for the full model and all component-removal configurations, except for the explicit zero-layer condition.

Nine configurations were examined:

1) **A0—Full AgriHGT:**
   All graph relations, metadata nodes, learnable node identities, message-passing layers, residual projection, and learned fusion gate were retained.
2) **A1—No HGT message passing:**
   The zero-layer result from the main DINOv3 depth experiment was reused. No HGT message-passing operation was performed.
3) **A2—No leaf-similarity edges:**
   The `similar_to` and reverse-`similar_to` relations were removed.
4) **A3—No crop knowledge:**
   Crop nodes and the `grown_on`, reverse-`grown_on`, `affects`, and reverse-`affects` relations were removed.
5) **A4—No disease taxonomy:**
   Disease-category nodes and the `is_type_of`, reverse-`is_type_of`, `co_category`, and reverse-`co_category` relations were removed.
6) **A5—No disease visual grounding:**
   The `has_disease` and reverse-`has_disease` relations were removed from the `train_msg` subset.
7) **A6—No learnable meta-node identity:**
   Learnable crop, disease, and disease-category embeddings were replaced with fixed zero vectors.
8) **A7—No residual feature bypass:**
   The direct DINOv3 residual projection was removed. Classification was performed using only the graph-propagated leaf representation.
9) **A8—No learned fusion gate:**
   The learned sigmoid gate was replaced with a fixed equal-weight average:

$$\mathbf{z}_i = 0.5\mathbf{r}_i + 0.5\mathbf{h}_i.$$

Configurations A0 and A2–A8 were trained for a maximum of 400 epochs with an early-stopping patience of 50 epochs. A hidden dimension of 256, four attention heads, HGT dropout of 0.2, AdamW with a learning rate of 1×10^{-3}, weighted cross-entropy, and cosine annealing were retained.

A1 was not retrained because the zero-layer DINOv3 result had already been produced during the main depth experiment. The architectural ablation was performed using one fixed seed and was therefore interpreted as a controlled component analysis rather than a multi-seed estimate of statistical stability.

Y. Evaluation Metrics

Performance was evaluated using classification accuracy and macro-averaged F1-score.

Accuracy was calculated as

$$\text{Accuracy} = \frac{\text{Number of correctly classified images}}{\text{Total number of evaluated images}}.$$

For class c, precision and recall were calculated as

$$P_c = \frac{TP_c}{TP_c + FP_c},$$

and

$$R_c = \frac{TP_c}{TP_c + FN_c}.$$

The class-specific F1-score was calculated as

$$F1_c = \frac{2P_cR_c}{P_c + R_c}.$$

Macro-F1 was calculated as

$$F1_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} F1_c,$$

where

$$C = 28$$

for the unified multi-crop experiment.

Per-class precision, recall, F1-score, and support were also reported. Support denotes the number of true test images belonging to a class. At least one validation image and one test image were retained for every class.

A confusion matrix was generated to examine the distribution of errors across crops and disease classes.

Single-seed results were reported for the seven backbone experiments, message-passing-depth comparisons, and DINOv3 architectural ablation. Mean and standard deviation were reported for the five-seed ResNet50 graph-component ablation and cross-crop scope experiment.

IV. Experimental Results

A. Experimental Setting

Seven visual backbones were evaluated: ResNet50, ConvNeXt-Tiny, EfficientNet-B3, Swin-Tiny, DenseNet121, DINOv2 ViT-B/14, and DINOv3 ViT-B/16. The final experimental design used the duplicate-aware, support-controlled split described in Section **??**, containing 25,122 training images, 896 validation images, and 848 test images. At least one validation image and one test image were retained for each of the 28 classes.

For every backbone, zero-, one-, and two-layer AgriHGT models were evaluated. Model depth and checkpoint selection were performed using validation accuracy. Test accuracy and test macro-F1 were calculated only after the selected checkpoint had been restored. The backbone, depth, and logistic-regression comparisons were based on one fixed-seed run for each configuration.

The ResNet50 graph-component and cross-crop scope experiments were evaluated using five seeds. Mean and standard deviation are reported for these experiments. The comprehensive DINOv3 architectural ablation was designed as a fixed-seed controlled experiment.

B. Backbone Fine-Tuning

Table III reports the highest validation accuracy obtained during backbone adaptation.

The highest fine-tuning validation accuracy was obtained with DINOv3 ViT-B/16, followed by ResNet50 and DINOv2 ViT-B/14. DenseNet121 produced the lowest fine-tuning validation accuracy. The ordering suggests that the quality of the extracted visual representation contributed substantially to the downstream graph-classification performance. However, the backbone result alone does not isolate the contribution of heterogeneous message passing.

TABLE III: Backbone Fine-Tuning Validation Accuracy

Backbone	Best Validation Accuracy
ResNet50	0.9241
ConvNeXt-Tiny	0.9029
EfficientNet-B3	0.8839
Swin-Tiny	0.8816
DenseNet121	0.8402
DINOv2 ViT-B/14	0.9096
DINOv3 ViT-B/16	**0.9286**

C. Selected AgriHGT Models

For each backbone, the HGT depth with the highest validation accuracy was selected. The resulting test performance is presented in Table IV.

The strongest selected-model result was obtained with DINOv3 ViT-B/16. A test accuracy of 0.9387 and a macro-F1 of 0.8588 were achieved. DINOv2 ViT-B/14 ranked second under both metrics, with a test accuracy of 0.9257 and a macro-F1 of 0.8374.

Among the conventional supervised backbones, the highest selected-model accuracy was obtained with ResNet50. ConvNeXt-Tiny achieved a lower accuracy than ResNet50 but a higher macro-F1. EfficientNet-B3 and Swin-Tiny produced similar accuracy levels, although higher macro-F1 was obtained with EfficientNet-B3. The weakest selected-model result was obtained with DenseNet121.

The two self-supervised DINO backbones therefore outperformed the supervised convolutional and hierarchical-transformer backbones under both accuracy and macro-F1. This result indicates that the pretrained DINO representations were particularly suitable for the visual relationships used by AgriHGT.

D. Message-Passing Depth Ablation

Zero-, one-, and two-layer models were evaluated for every backbone. The zero-layer configuration retained the type-specific input projections, learnable meta-node embeddings, gated residual path, and compatibility decoder, but no HGT message passing was performed. Table V reports the complete depth comparison. The asterisk identifies the configuration selected using validation accuracy.

The effect of message passing differed across the visual backbones. For DINOv3, progressive improvements were obtained as the depth increased from zero to two layers. Relative to the zero-layer configuration, the selected two-layer model increased test accuracy by approximately 2.12 percentage points and macro-F1 by approximately 4.69 percentage points.

For DINOv2, the selected one-layer model improved test accuracy by approximately 1.77 percentage points and macro-F1 by approximately 5.62 percentage points relative to the zero-layer model. The one- and two-layer configurations produced the same validation accuracy, but the one-layer model was retained because the first configuration reaching the highest validation accuracy was selected.

Substantial improvements were also observed for DenseNet121 and ConvNeXt-Tiny. The selected DenseNet121 model improved test accuracy by approximately 6.02

TABLE IV: Validation-Selected AgriHGT Performance Across Visual Backbones

Backbone	Selected Layers	Validation Accuracy	Test Accuracy	Test Macro-F1
ResNet50	1	**0.9330**	0.9068	0.7986
ConvNeXt-Tiny	2	0.9129	0.8998	0.8071
EfficientNet-B3	1	0.9029	0.8915	0.7973
Swin-Tiny	2	0.9128	0.9055	0.7828
DenseNet121	1	0.8704	0.8548	0.7616
DINOv2 ViT-B/14	1	0.9219	0.9257	0.8374
DINOv3 ViT-B/16	2	0.9263	**0.9387**	**0.8588**

TABLE V: Message-Passing Depth Ablation. The Asterisk Identifies the Configuration Selected by Validation Accuracy

Backbone	Layers	Validation Accuracy	Test Accuracy	Test Macro-F1
ResNet50	0	0.9252	0.9057	0.8114
ResNet50	1*	**0.9330**	0.9068	0.7986
ResNet50	2	0.9319	**0.9163**	**0.8224**
ConvNeXt-Tiny	0	0.9007	0.8774	0.7877
ConvNeXt-Tiny	1	0.9107	**0.9057**	**0.8297**
ConvNeXt-Tiny	2*	**0.9129**	0.8998	0.8071
EfficientNet-B3	0	0.8828	0.8868	0.8051
EfficientNet-B3	1*	**0.9029**	0.8915	0.7973
EfficientNet-B3	2	0.9007	**0.9033**	**0.8295**
Swin-Tiny	0	0.8793	0.8843	0.7792
Swin-Tiny	1	0.9039	0.9032	**0.8046**
Swin-Tiny	2*	**0.9128**	**0.9055**	0.7828
DenseNet121	0	0.8168	0.7946	0.7041
DenseNet121	1*	**0.8704**	**0.8548**	**0.7616**
DenseNet121	2	0.8570	0.8489	0.7549
DINOv2 ViT-B/14	0	0.9141	0.9080	0.7812
DINOv2 ViT-B/14	1*	**0.9219**	**0.9257**	**0.8374**
DINOv2 ViT-B/14	2	**0.9219**	0.9198	0.8008
DINOv3 ViT-B/16	0	0.9107	0.9175	0.8119
DINOv3 ViT-B/16	1	0.9241	0.9328	0.8345
DINOv3 ViT-B/16	2*	**0.9263**	**0.9387**	**0.8588**

percentage points and macro-F1 by approximately 5.75 percentage points relative to zero layers. The selected ConvNeXt-Tiny model improved accuracy by approximately 2.24 percentage points and macro-F1 by approximately 1.94 percentage points.

Validation-based selection did not always identify the configuration with the highest test macro-F1. The two-layer ResNet50 model, one-layer ConvNeXt-Tiny model, two-layer EfficientNet-B3 model, and one-layer Swin-Tiny model produced stronger test macro-F1 than their corresponding validation-selected configurations. These values are reported as a depth-sensitivity analysis and were not used to revise the original selection rule after test evaluation.

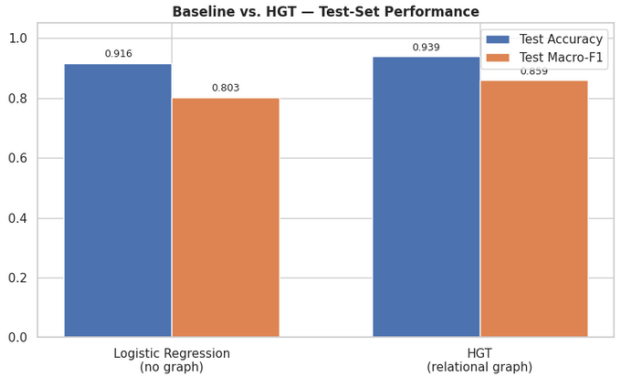


Fig. 3: AgriHGT vs Logistic Regression

E. Comparison with Feature-Only Logistic Regression

A multinomial logistic-regression classifier was trained using the same standardized feature matrix generated by each backbone. Fig 3 illustrates the comparison between AgriHGT model and logistic regression. Table VI compares the feature-only baseline with the AgriHGT configuration selected using validation accuracy.

The clearest improvement over the feature-only baseline was obtained with DINOv3. Test accuracy increased by approximately 2.24 percentage points, while macro-F1 increased by approximately 5.61 percentage points. DINOv2 also benefited from the graph, with increases of approximately 0.83 percentage points in accuracy and 3.05 percentage points in macro-F1.

For ConvNeXt-Tiny, the same rounded test accuracy was obtained by both models, while macro-F1 increased slightly under AgriHGT. For ResNet50 and DenseNet121, small accuracy improvements were obtained, but macro-F1 remained approximately unchanged or decreased slightly.

For EfficientNet-B3, logistic regression exceeded the selected AgriHGT model under both test metrics. Swin-Tiny produced approximately the same test accuracy under both models, but the logistic-regression baseline achieved a higher

TABLE VI: Validation-Selected AgriHGT and Feature-Only Logistic Regression

Backbone	Model	Validation Accuracy	Test Accuracy	Test Macro-F1
ResNet50	Logistic Regression	0.9141	0.9057	**0.7997**
ResNet50	AgriHGT	**0.9330**	**0.9068**	0.7986
ConvNeXt-Tiny	Logistic Regression	0.8996	0.8998	0.8064
ConvNeXt-Tiny	AgriHGT	**0.9129**	0.8998	**0.8071**
EfficientNet-B3	Logistic Regression	0.8862	**0.8998**	**0.7996**
EfficientNet-B3	AgriHGT	**0.9029**	0.8915	0.7973
Swin-Tiny	Logistic Regression	0.8894	**0.9055**	**0.8045**
Swin-Tiny	AgriHGT	**0.9128**	0.9055	0.7828
DenseNet121	Logistic Regression	0.8514	0.8512	**0.7619**
DenseNet121	AgriHGT	**0.8704**	**0.8548**	0.7616
DINOv2 ViT-B/14	Logistic Regression	0.8951	0.9175	0.8069
DINOv2 ViT-B/14	AgriHGT	**0.9219**	**0.9257**	**0.8374**
DINOv3 ViT-B/16	Logistic Regression	0.9040	0.9163	0.8027
DINOv3 ViT-B/16	AgriHGT	**0.9263**	**0.9387**	**0.8588**

macro-F1.

A universal advantage over feature-only classification was therefore not observed. The graph contribution depended on the visual representation and the depth selected through validation accuracy. The strongest and most consistent graph benefits were obtained with the two self-supervised DINO backbones.

The logistic-regression comparison does not isolate message passing completely because metadata nodes, learnable node identities, residual fusion, and the compatibility decoder are also absent from the baseline. The zero-layer comparison in Section IV-D provides the more controlled assessment of heterogeneous message propagation.

F. Per-Class Performance of DINOv3 AgriHGT

The selected two-layer DINOv3 model was examined at class level because the strongest overall test accuracy and macro-F1 were obtained from this configuration. Table VII reports the complete class-level results.

F1-scores of 1.000 were obtained for Bottle Gourd Healthy, Tomato White Spot, Bottle Gourd Fungal Damage Leaf, and several other strongly separated classes. Most Bottle Gourd, Papaya, and Tomato conditions achieved F1-scores above 0.90.

The weakest performance remained concentrated among Zucchini conditions. Zucchini Angular Leaf Spot had 36 test images but achieved a recall of 0.361 and an F1-score of 0.473. The difficulty of this class therefore cannot be attributed only to extremely low support. Zucchini Anthracnose produced an F1-score of 0.381 despite having ten test examples.

The rarest Zucchini classes contained only one or two test examples. Their class-specific scores are consequently unstable and can change substantially after a single correct or incorrect prediction.

The confusion matrix remained approximately block diagonal with respect to crop identity. Most errors occurred between visually related disease conditions belonging to the same crop. Cross-crop classification errors were less frequent than within-crop disease confusions.

G. DINOv3 Training Dynamics

The selected DINOv3 model reached its highest validation accuracy of 0.9263 during the early part of training. No further improvement above the minimum threshold was obtained for 50 consecutive epochs, and training was stopped at epoch 90. The restored checkpoint achieved a test accuracy of 0.9387 and a macro-F1 of 0.8588.

The early-stopping behaviour indicates that the complete 400-epoch budget was not required for the selected DINOv3 configuration. The reported result was calculated from the checkpoint with the highest validation accuracy rather than from the final epoch.

H. Cross-Crop and Crop-Specific Graph Comparison

The complete multi-crop graph was compared with independently trained crop-specific graphs. Mean and standard deviation across five seeds are reported in Table VIII.

For Bottle Gourd, the cross-crop graph increased mean accuracy by approximately 0.63 percentage points and macro-F1 by approximately 0.58 percentage points. For Tomato, increases of approximately 1.53 percentage points in accuracy and 1.45 percentage points in macro-F1 were obtained.

Almost no difference was observed for Papaya. The same mean accuracy was obtained under both graph scopes, while the macro-F1 difference was approximately 0.04 percentage points.

For Zucchini, lower performance was obtained from the cross-crop graph. Accuracy decreased by approximately 1.06 percentage points, while macro-F1 decreased by approximately 1.26 percentage points relative to the crop-specific model.

A uniform benefit from cross-crop structure was therefore not observed. Positive effects were obtained for Bottle Gourd and Tomato, almost no effect was obtained for Papaya, and a negative effect was obtained for Zucchini. The contribution of shared cross-crop structure was therefore crop-dependent.

I. DINOv3 Architectural Ablation

The DINOv3 architectural ablation was designed to evaluate the full AgriHGT model and eight component-removal configurations, including the removal of HGT message passing, image-similarity edges, crop knowledge, disease taxonomy, disease visual grounding, learnable meta-node identities, the residual bypass, and the learned fusion gate.

TABLE VII: Per-Class Test Performance of the Selected Two-Layer DINOv3 AgriHGT Model

Class	Support	Precision	Recall	F1
Papaya_Curled_Yellow_Spot	66	1.000	0.939	0.969
Papaya_Carica_Insect_Hole	56	0.963	0.929	0.945
Bottle_Gourd_Mosaic_Virus	52	0.962	0.981	0.971
Bottle_Gourd_Downy_Mildew	51	0.980	0.980	0.980
Papaya_Pathogen_Symptoms	51	0.981	1.000	0.990
Bottle_Gourd_Healthy	49	1.000	1.000	1.000
Tomato_Healthy	47	0.979	0.979	0.979
Bottle_Gourd_Alternaria_Leaf_Blight	46	0.978	0.978	0.978
Tomato_Spot	43	1.000	0.907	0.951
Tomato_Mosaic	40	0.952	1.000	0.976
Bottle_Gourd_Anthracnose	40	1.000	0.950	0.974
Papaya_Healthy_Leaf	36	0.947	1.000	0.973
Zucchini_Angular_Leaf_Spot	36	0.684	0.361	0.473
Bottle_Gourd_Early_Alternaria_Leaf_Blight	31	0.969	1.000	0.984
Papaya_Bacterial_Blight	31	0.969	1.000	0.984
Papaya_Mosaic_Virus	30	0.968	1.000	0.984
Zucchini_Yellow_Mosaic_Virus	30	0.967	0.967	0.967
Zucchini_Downy_Midew	23	0.909	0.870	0.889
Tomato_White_spot	21	1.000	1.000	1.000
Tomato_Downy	19	0.905	1.000	0.950
Bottle_Gourd_Fungal_Damage_Leaf	15	1.000	1.000	1.000
Papaya_Yellow_Necrotic_Spots_Holes	11	0.917	1.000	0.957
Zucchini_Anthracnose	10	0.364	0.400	0.381
Zucchini_Insect_Damage	8	0.533	1.000	0.696
Zucchini_Healthy	2	1.000	0.500	0.667
Zucchini_Dry_Leaf	2	0.222	1.000	0.364
Zucchini_Iron_Chlorosis_Damage	1	0.250	1.000	0.400
Zucchini_Xanthomonas_Leaf_Spot	1	0.500	1.000	0.667

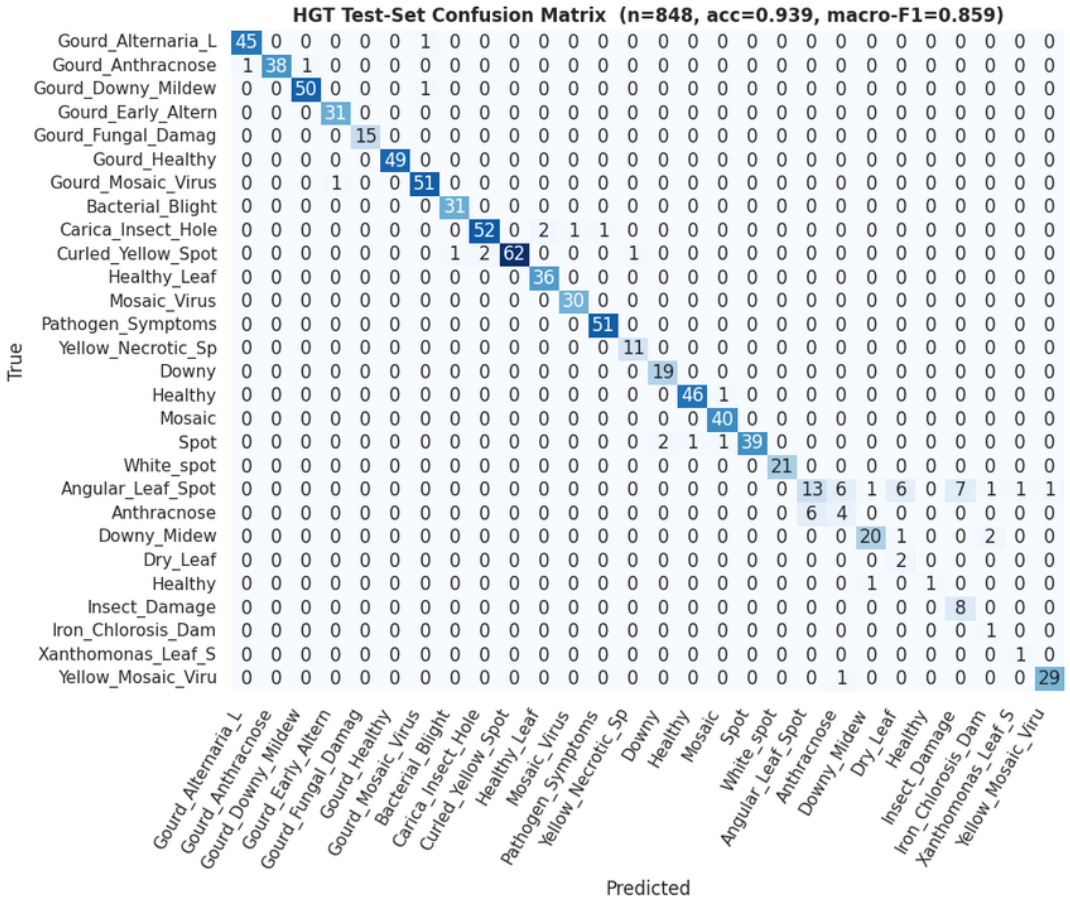


Fig. 4: Test confusion matrix for the selected two-layer DINOv3 AgriHGT model. The support-controlled test partition contains 848 images; test accuracy is 0.939 and macro-F1 is 0.859.

J. Results Summary

The strongest overall AgriHGT result was obtained using DINOv3 ViT-B/16 with two HGT layers. A test accuracy of

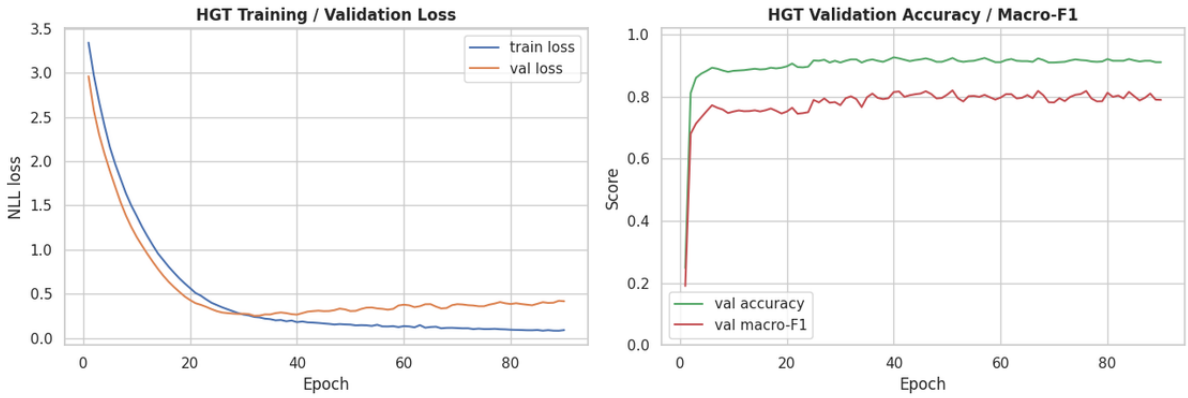


Fig. 5: Training loss, validation loss, validation accuracy, and validation macro-F1 for the selected two-layer DINOv3 AgriHGT model.

TABLE VIII: Cross-Crop and Crop-Specific Graph Performance

Crop	Graph Scope	Accuracy	Accuracy SD	Macro-F1	Macro-F1 SD
Bottle Gourd	Cross-crop	**0.9451**	0.0019	**0.9505**	0.0017
Bottle Gourd	Crop-specific	0.9387	0.0053	0.9447	0.0048
Papaya	Cross-crop	0.9167	0.0065	**0.9252**	0.0051
Papaya	Crop-specific	0.9167	0.0019	0.9248	0.0030
Tomato	Cross-crop	**0.9494**	0.0067	**0.9538**	0.0063
Tomato	Crop-specific	0.9341	0.0049	0.9392	0.0047
Zucchini	Cross-crop	0.5735	0.0268	0.4052	0.0359
Zucchini	Crop-specific	**0.5841**	0.0177	**0.4178**	0.0196

0.9387 and a macro-F1 of 0.8588 were achieved. DINOv2 ViT-B/14 ranked second, reaching 0.9257 accuracy and 0.8374 macro-F1.

The two DINO backbones also produced the clearest improvements over feature-only logistic regression. For DINOv3, AgriHGT improved accuracy by approximately 2.24 percentage points and macro-F1 by approximately 5.61 percentage points. For DINOv2, the corresponding improvements were approximately 0.83 and 3.05 percentage points.

Message passing improved the strongest observed test result relative to the zero-layer model for every backbone. However, selection by validation accuracy did not always identify the depth with the highest test macro-F1. This difference was most visible for ResNet50, ConvNeXt-Tiny, EfficientNet-B3, and Swin-Tiny.

The multi-seed ResNet50 ablation showed that image-similarity edges made the clearest contribution to overall accuracy. The contribution of disease-category structure was smaller and depended on the evaluation metric. The cross-crop comparison showed positive effects for Bottle Gourd and Tomato, almost no effect for Papaya, and a negative effect for Zucchini.

The results therefore support a backbone-dependent and crop-dependent interpretation of relational modelling. The strongest benefit was obtained when high-quality self-supervised DINO features were combined with heterogeneous message passing, while consistent improvements were not observed for every backbone or every crop.

References

[1] P. Srivastava, K. Mishra, V. Awasthi, V. K. Sahu, and P. K. Pal, "Plant disease detection using convolutional neural network," *International Journal of Advanced Research*, vol. 9, no. 1, pp. 691–698, 2021. DOI: 10.21474/IJAR01/12346.

[2] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018. DOI: 10.1016/j.compag.2018.01.009.

[3] S. Murugavalli and R. Gopi, "Plant leaf disease detection using vision transformers for precision agriculture," *Scientific Reports*, vol. 15, p. 22 361, 2025. DOI: 10.1038/s41598-025-05102-0.

[4] R. A. Boukabouya, A. Moussaoui, and M. Berrimi, "Vision transformer based models for plant disease detection and diagnosis," in *2022 5th International Symposium on Informatics and its Applications (ISIA)*, 2022, pp. 1–6. DOI: 10.1109/ISIA55826.2022.9993508.

[5] W. Liu and A. Zhang, "Plant disease detection algorithm based on efficient swin transformer," *Computers, Materials & Continua*, vol. 82, no. 2, pp. 3045–3068, 2025. DOI: 10.32604/cmc.2024.058640.

[6] K. Ananthajothi and K. Lokesh, "Graph neural networks for smarter plant disease diagnosis and classification," in *2026 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and*

Electronics (IITCEE), Bangalore, India, 2026. DOI: 10.1109/IITCEE67948.2026.11394113.

[7] M. Elumalai, M. Annamalai, and D. M, "Attention-enhanced gnn model for fungal disease classification in spinach leaves using monospectral imaging," *Scientific Reports*, 2026, Published online ahead of issue/volume assignment. DOI: 10.1038/s41598-026-54574-1.

[8] P. Kalpana et al., "An ensemble heterogeneous transformer model for an effective diagnosis of multiple plant diseases," *Frontiers in Plant Science*, vol. 16, p. 1 693 095, 2025. DOI: 10.3389/fpls.2025.1693095.

[9] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proceedings of The Web Conference 2020 (WWW '20)*, Taipei, Taiwan: ACM, 2020, pp. 2704–2710. DOI: 10.1145/3366423.3380027.

[10] S. Parez, N. Dilshad, N. S. Alghamdi, T. M. Alanazi, and J. W. Lee, "Visual intelligence in precision agriculture: Exploring plant disease detection via efficient vision transformers," *Sensors*, vol. 23, no. 15, p. 6949, 2023. DOI: 10.3390/s23156949.

[11] G. Shandilya, S. Gupta, H. G. Mohamed, S. Bharany, A. U. Rehman, and S. Hussen, "Enhanced maize leaf disease detection and classification using an integrated cnn-vit model," *Food Science & Nutrition*, vol. 13, no. 7, e70513, 2025. DOI: 10.1002/fsn3.70513.

[12] P. S. Roy and V. Kukreja, "Vision transformers for rice leaf disease detection and severity estimation: A precision agriculture approach," *Journal of the Saudi Society of Agricultural Sciences*, vol. 24, p. 3, 2025. DOI: 10.1007/s44447-025-00007-w.

[13] A. Bera, D. Bhattacharjee, and O. Krejcar, "Pnd-net: Plant nutrition deficiency and disease classification using graph convolutional network," *Scientific Reports*, vol. 14, p. 15 537, 2024. DOI: 10.1038/s41598-024-66543-7.

[14] A. Bajpai, Y. Pal, S. Sahu, and V. Srivastava, "Enhanced potato leaf disease detection through multi-modal fusion of graph neural networks and resnet18," in *2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)*, 2024, pp. 1498–1504. DOI: 10.1109/CICN63059.2024.10847571.

[15] S. U. M. Rao, K. Sreekala, P. S. Rao, N. Shirisha, G. Srinivas, and E. Sreedevi, "Plant disease classification using novel integration of deep learning cnn and graph convolutional networks," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 36, no. 3, pp. 1721–1730, 2024. DOI: 10.11591/ijeecs.v36.i3.pp1721-1730.

[16] B. Surekha and T. S. M. Rao, "Adjleafgnn: A hybrid deep learning and graph neural network framework for probabilistic modeling of adjacent leaf disease spread in precision agriculture," *Scientific Reports*, 2026, Published online ahead of issue/volume assignment. DOI: 10.1038/s41598-026-45061-8.

[17] C. Li, S. Yang, D. Liang, P. Chen, and W. Dong, "Construction of a multi-source, heterogeneous rice disease and pest knowledge graph based on the marbc model," *Agronomy*, vol. 15, no. 3, p. 566, 2025. DOI: 10.3390/agronomy15030566.

[18] X. Zhao et al., "Implementation of large language models and agricultural knowledge graphs for efficient plant disease detection," *Agriculture*, vol. 14, no. 8, p. 1359, 2024. DOI: 10.3390/agriculture14081359.

[19] S. Lin, Y. Xiu, J. Kong, C. Yang, and C. Zhao, "An effective pyramid neural network based on graph-related attentions structure for fine-grained disease and pest identification in intelligent agriculture," *Agriculture*, vol. 13, no. 3, p. 567, 2023. DOI: 10.3390/agriculture13030567.

[20] M. M. Billah, M. A. Rahman, S. Sagor, S. Parvin, and M. S. Uddin, *Agri-vision4: A comprehensive multi-crop leaf disease dataset of tomato, papaya, zucchini, and bottle gourd from bangladesh*, version 1, Mendeley Data, 2026. DOI: 10.17632/8t6k37ztxc.1.